EgoTECH

PROJECT HANDOVER DOCUMENT

EgoTECH PDF Vault — Document & Image Storage System

Document Type	Project Handover Document (PHD)
Project Name	EgoTECH PDF Vault
Organization	EgoTECH
Version	1.0
Date	October 20, 2026

1. Introduction

1.1 Purpose

This Project Handover Document (PHD) formally records the transfer of the EgoTECH PDF Vault application from the EgoTECH development team to the Client's operational or maintenance team. It consolidates all information required for the receiving team to take ownership of, operate, maintain, and further develop the system without reliance on the original developers.

1.2 Handover Summary

Field	Details
Project Name	EgoTECH PDF Vault
Version Handed Over	1.0
Handover Date	October 20, 2026
Handing Over Party	EgoTECH — Development Team
Receiving Party	_____
Project Manager	_____
Technical Lead (EgoTECH)	_____
Technical Lead (Client)	_____
Repository URL	_____

1.3 Project Background

EgoTECH PDF Vault is a full-stack, self-hosted document management system. It provides authenticated users with the ability to upload, organize, search, download, and delete PDF documents and image assets through a modern React-based web interface backed by a Python FastAPI REST API and an SQLite relational database.

The application was designed to address the need for a lightweight, privacy-conscious document storage solution that can be deployed on-premises without reliance on commercial cloud storage platforms.

2. Project Deliverables

2.1 Software Deliverables

Deliverable	Location / Notes	Status
Backend source code	PDF-Storage-System/backend/	Complete
Frontend source code	PDF-Storage-System/frontend/	Complete
SQLite database	backend/storage/pdf_vault.db (auto-created)	Complete
Storage directory	backend/storage/pdfs/ (PDFs + images)	Complete
Environment config	backend/.env (from .env.example template)	Client must configure
Git repository	Full history on main and dev branches	Complete
requirements.txt	backend/requirements.txt	Complete

package.json	frontend/package.json	Complete

2.2 Documentation Deliverables

Document	File	Status
Software Project Proposal (SPP)	docs/Software_Project_Proposal.docx	Complete
Software Requirements Spec (SRS)	docs/Software_Requirements_Specification.docx	Complete
User Manual (UM)	docs/User_Manual.docx	Complete
Deployment Checklist	docs/Deployment_Checklist.docx	Complete
Maintenance Agreement (SMA)	docs/Maintenance_Agreement.docx	Complete
Project Handover Document (PHD)	docs/Project_Handover_Document.docx	This document

3. System Architecture Overview

3.1 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18.3.1	UI framework (SPA)
Build Tool	Vite	6.0.5	Frontend bundler & dev server
Icons	Lucide-React	0.468.0	Icon library
Backend	Python / FastAPI	0.153.0	REST API server
ASGI Server	Uvicorn (with standard extras)	0.54.0	Application server
ORM	SQLAlchemy	2.1.5	Database abstraction
Validation	Pydantic + Pydantic-Settings	2.14.1 / 2.16.1	Schema validation & config
Auth	JWT (via FastAPI security)	Built-in	Stateless auth tokens
File Upload	python-multipart	0.0.32	Multipart form parsing
Database	SQLite 3	Built-in	Relational data storage

3.2 Directory Structure

```
PDF-Storage-System/
  backend/
    app/
      auth/           # JWT auth: router.py, security.py
      routers/        # API routes: documents.py, images.py
      config.py       # Pydantic-Settings config
      database.py     # SQLAlchemy engine & session
      main.py         # FastAPI app factory & lifespan
```

```
      models.py       # ORM models: User, Folder, Document, ImageCategory,
ImageAsset
    storage/
      pdf_vault.db    # SQLite database (auto-created)
      pdfs/           # Uploaded PDF files (UUID-named)
        images/       # Uploaded image files
    .env              # Runtime configuration (must not be committed)
    .env.example      # Configuration template
    requirements.txt  # Python dependencies
  frontend/
    src/
      components/     # React components (Sidebar, DocumentCard, UploadPanel,
etc.)
      services/       # API client functions (api.js, authApi.js)
      styles/         # CSS modules (app.css, folders.css, images.css, etc.)
      App.jsx         # Root application component
      main.jsx        # React DOM entry point
    index.html        # HTML entry point
    vite.config.js    # Vite build configuration
    package.json      # npm dependencies & scripts
  docs/               # All project documentation (Word files)
```

3.3 Database Schema Summary

Table	Primary Key	Foreign Keys	Key Columns
users	id (UUID)	—	username, email, password_hash, created_at
folders	id (UUID)	user_id → users	name, created_at
documents	id (UUID)	user_id → users, folder_id → folders	filename, stored_filename, size_bytes, created_at
image_categories	id (UUID)	user_id → users	name, created_at
image_assets	id (UUID)	user_id → users, category_id → image_categories	filename, stored_filename, content_type, size_bytes, created_at

3.4 API Overview

All API routes are prefixed with /api/v1 (configurable). Full interactive documentation is available at http://<server>:8000/docs.

Module	Endpoints	Auth Required
Authentication	/auth/register, /auth/login, /auth/me	register & login: No; /me: Yes
Documents	/documents (GET, POST),	Yes

	/documents/{id}/download, /documents/{id} (DELETE)	
Folders	/documents/folders (GET, POST), /documents/folders/{id} (DELETE)	Yes
Images	/images (GET, POST), /images/{id}/file, /images/{id} (DELETE)	Yes
Image Categories	/images/categories (GET, POST), /images/categories/{id} (DELETE)	Yes

4. Environment Configuration

The backend is configured entirely via the backend/.env file. The receiving team must:

1. Copy backend/.env.example to backend/.env.
2. Set AUTH_SECRET to a strong, unique random string (minimum 32 characters). This is critical for JWT security.
3. Set CORS_ORIGINS to the production frontend URL(s).
4. Review and adjust MAX_UPLOAD_MB and AUTH_TOKEN_MINUTES as required.
5. Ensure STORAGE_DIR points to a writable directory with sufficient disk space.

Variable	Default	Must Change for Production?
AUTH_SECRET	(placeholder)	YES — critical security setting
CORS_ORIGINS	http://localhost:5173	YES — set to production frontend URL
DATABASE_URL	sqlite:///./storage/pdf_vault.db	Recommended for production: PostgreSQL
STORAGE_DIR	storage/pdfs	Set to absolute path on server
MAX_UPLOAD_MB	25	Adjust per business requirements
AUTH_TOKEN_MINUTES	60	Adjust per security policy

5. Application Startup Commands

5.1 Backend

```
cd PDF-Storage-System/backend
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000
```

For production (no auto-reload, bound to all interfaces):

```
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

5.2 Frontend (Development)

```
cd PDF-Storage-System/frontend
npm install
```

```
npm run dev
```

5.3 Frontend (Production Build)

```
npm run build
```

Serve the generated dist/ directory using Nginx, Apache, IIS, or any static file server.

6. Known Issues & Limitations

ID	Issue / Limitation	Severity	Planned Fix
KI-01	Free-plan upload quota enforced client-side only (localStorage). A determined user could bypass it via direct API calls.	Medium	v2.0 — server-side quota enforcement
KI-02	Plan activation (Free → Pro) is session-local; resets on logout. No persistent billing integration.	Medium	v2.0 — billing & persistent plan storage
KI-03	SQLite is not suitable for high-concurrency or large-scale production use.	Medium	v2.0 — PostgreSQL migration path prepared
KI-04	No in-browser PDF preview; documents are download-only.	Low	v2.0 — PDF.js integration planned
KI-05	No admin dashboard for user management or system-wide statistics.	Low	v2.0 — Admin module planned
KI-06	Password reset / forgot-password flow not implemented.	Low	v2.0 — Email-based reset planned
KI-07	No email verification on registration.	Low	v2.0 — Email verification planned
KI-08	HTTPS not configured in default setup; must be set up separately for production.	High	Deployment team responsibility (see Deployment Checklist)

7. Pending Tasks & Backlog

The following items were identified during development but deferred to a future version:

Priority	Task	Assigned To
High	Configure HTTPS / TLS for production deployment	Receiving team
High	Move plan quota enforcement to the backend	EgoTECH (v2.0)

High	Implement persistent Pro plan storage in DB	EgoTECH (v2.0)
Medium	Migrate database from SQLite to PostgreSQL	EgoTECH (v2.0)
Medium	Add server-side pagination for large document lists	EgoTECH (v2.0)
Medium	Implement password reset via email	EgoTECH (v2.0)
Medium	Add email verification on registration	EgoTECH (v2.0)
Medium	Build admin dashboard for user & storage management	EgoTECH (v2.0)
Low	Add in-browser PDF preview (PDF.js)	EgoTECH (v2.0)
Low	Add bulk upload / download support	EgoTECH (v2.0)
Low	Implement folder sharing between users	EgoTECH (v2.0)
Low	Add dark/light theme toggle	EgoTECH (v2.0)

8. Credentials & Access Handover

The following credentials and access items are being transferred to the receiving team. All credentials should be rotated immediately after handover.

Item	Notes	Transferred?
Git repository access	Receiving team added as repository owners	[ ] Yes [ ] No
Server SSH / RDP access	Credentials provided separately via secure channel	[ ] Yes [ ] No
Backend .env file	Contains AUTH_SECRET — share via encrypted channel	[ ] Yes [ ] No
Database backup	Latest backup provided as .db file	[ ] Yes [ ] No
Storage backup	Latest pdfs/ directory backup provided	[ ] Yes [ ] No
API documentation URL	http://<server>:8000/docs	[ ] Yes [ ] No
Issue tracker access	GitHub Issues or Jira board shared	[ ] Yes [ ] No
Support email	support@egotech.com	[ ] Yes [ ] No

9. Training & Knowledge Transfer

Post-Handover Support

EgoTECH will provide up to two weeks of post-handover warranty support at no additional charge. During this period, the receiving team may contact EgoTECH for clarifications, bug fixes for issues

present in the handed-over version, and guidance on configuration. After this period, support is governed by the Maintenance Agreement.

10. Handover Acceptance Criteria

The handover is considered complete and accepted when all of the following conditions are met:

#	Checklist Item	Status
1	All software deliverables listed in Section 2.1 have been received by the Client team.	[ ] Pending
2	All documentation deliverables listed in Section 2.2 have been received.	[ ] Pending
3	All credentials in Section 8 have been transferred and rotated.	[ ] Pending
4	The Deployment Checklist has been completed and signed off on the target environment.	[ ] Pending
5	All knowledge transfer sessions listed in Section 9.1 have been conducted.	[ ] Pending
6	The receiving team has successfully started both backend and frontend servers independently.	[ ] Pending
7	Smoke tests (register, login, upload PDF, download PDF, upload image, logout) pass on the production environment.	[ ] Pending
8	The Maintenance Agreement is signed and in effect.	[ ] Pending
9	Outstanding issues in Section 6 have been documented and communicated.	[ ] Pending
10	The receiving team confirms they have no outstanding questions that would block independent operation.	[ ] Pending

11. Handover Sign-Off

By signing below, both parties confirm that the project handover has been completed satisfactorily and all acceptance criteria have been met.

HANDING OVER PARTY — EgoTECH

Field	Details
Authorized Representative	_____
Title	_____
Signature	_____
Date	_____

RECEIVING PARTY — Client

Field	Details
Organization	_____
Authorized Representative	_____

Title	_____
Signature	_____
Date	_____